S.G.R.S.I
Sistema de Gestión de Recursos y Soporte de Informática

PROYECTO DE PASAJE DE GRADO

BT DE INFORMÁTICA

Edición 2026

Deklan Enterprice

Rol	Apellido	Nombre	C.I	Correo electrónico
Coordinador	González	Julián	5.742.892-8	julian.gonzalez.arrascaeta@gmail.com
Sub-Coordinador	Cuello	Santiago	5.731.237-3	augustocrafter22@gmail.com
Integrante 1	Forcelledo	Ezequiel	5.724.706-7	ezequielforcelledo9@gmail.com
Integrante 2	Venis	Federico	6.486.057-3	fede.venis@gmail.com

Docentes y unidades curriculares

Unidad curricular	Apellido	Nombre
Programación Full Stack	Lopez	Leandro
Administración de Sistemas Operativos	Sandler	Gerardo
Ingeniería de Software	Mederos	Marcela
Tutoría de Proyecto Utulab	Mederos	Marcela
Ciberseguridad	Venturino	Nibia

Tercera entrega

Fecha de entrega: 23/10/2026

1. PRESENTACIÓN DEL PROYECTO.....5
EQUIPO Y DOCENTES..... 5
RESUMEN DEL TRABAJO.....5
INTRODUCCIÓN Y CONTEXTO.....6
OBJETIVO GENERAL.....6
OBJETIVOS ESPECÍFICOS.....7
ALCANCE.....7
LIMITACIONES.....8
ESTRUCTURA DEL DOCUMENTO ORIGINAL.....8
2. TUTORÍA DE PROYECTO UTULAB.....8
2.1 PRIMERA ENTREGA: DIAGNÓSTICO.....8
ENTREVISTA AL EQUIPO DE SOPORTE.....8
OBSERVACIÓN DIRECTA NO PARTICIPANTE.....10
USUARIOS Y ACTORES.....10
PROBLEMA CENTRAL.....10
FODA DEL EQUIPO.....11
TABLA DE DECISIÓN POR ROL.....11
PRIMERA PROPUESTA DE SOLUCIÓN.....11
REPRESENTACIÓN INICIAL Y TAREAS.....12
ORGANIZACIÓN DEL EQUIPO.....12
2.2 SEGUNDA ENTREGA: PROTOTIPO Y VALIDACIÓN..... 13
2.3 TERCERA ENTREGA: CIERRE.....14
PROTOTIPO FINAL.....14
VALIDACIÓN FINAL.....14
IDEA INICIAL VS. SOLUCIÓN FINAL.....15
INFORME REFLEXIVO.....16
IMPACTO DEL PROYECTO.....16
COMUNICACIÓN Y MERCHANDISING FINAL.....17
3. INGENIERÍA DE SOFTWARE.....17
3.1 RELEVAMIENTO.....18
3.2 FACTIBILIDAD.....18
3.3 REQUERIMIENTOS FUNCIONALES (61).....19
3.4 REQUERIMIENTOS NO FUNCIONALES (28).....20
3.5 FODA DEL SISTEMA.....20
3.6 PLANIFICACIÓN Y MODELO DE DESARROLLO.....21

3.7 CASOS DE USO....22
3.8 DIAGRAMAS, RIESGOS Y FODA PONDERADO.... 24
3.9 MÉTRICAS Y COSTO-BENEFICIO.... 25
3.10 TESTING.... 27
3.11 TABLAS DE DECISIÓN....29
3.12 MANUAL DE USUARIO.... 30
3.13 MANUAL DE INSTALACIÓN Y MANTENIMIENTO.... 30
4. ADMINISTRACIÓN DE SISTEMAS OPERATIVOS....31
4.1 INSTALACIÓN DE UBUNTU SERVER.... 31
4.2 ROCKY LINUX EN VIRTUALBOX.... 32
4.3 ELECCIÓN DEL SISTEMA OPERATIVO....33
4.4 RED, SSH Y PAQUETES.... 33
4.5 ENTORNO LAMP....34
4.6 SEGURIDAD Y RESPALDOS.... 34
4.7 SCRIPT FINAL DEL SERVIDOR....35
4.8 CONTENEDORES DOCKER....37
4.9 SERVIDOR DE RESPALDO.... 38
5. PROGRAMACIÓN FULL STACK.... 40
5.1 HERRAMIENTAS....40
5.2 ESTÁNDARES DE CODIFICACIÓN....41
5.3 REPOSITORIO, RAMAS Y VERSIONADO....42
5.4 PERSISTENCIA Y NORMALIZACIÓN.... 42
5.5 PATRONES DE DISEÑO....44
5.6 API.... 44
5.7 REPOSITORIO Y ANEXOS DE CÓDIGO....46
6. CIBERSEGURIDAD.... 47
6.1 POLÍTICAS DE SEGURIDAD DE DEKLAN ENTERPRISE. 47
6.2 MARCO NORMATIVO URUGUAYO....48
7. CONCLUSIONES GENERALES.... 48
7.1 REFLEXIÓN FINAL DEL EQUIPO.... 49
8. GLOSARIO.... 49
9. BIBLIOGRAFÍA Y ANEXOS.... 51

1. PRESENTACIÓN DEL PROYECTO

El SGRSI (Sistema de Gestión de Recursos y Soporte de Informática) es una plataforma web interna que reemplaza planillas en papel, avisos verbales y correos por un registro único con seguimiento. Lo desarrolló Deklan Enterprise (grupo 3MB, ITI, UTU) como Proyecto de Pasaje de Grado 2026. Tercera entrega: 23/10/2026.

EQUIPO Y DOCENTES

Rol	Integrante	Área de referencia
Coordinador	Julián González	Programación / supervisión
Sub-coordinador	Santiago Cuello	Ingeniería de Software
Integrante 1	Ezequiel Forcelledo	UTULAB
Integrante 2	Federico Venis	Sistemas Operativos

Unidad curricular	Docente
Programación Full Stack	Leandro Lopez
Administración de Sistemas Operativos	Gerardo Sandler
Ingeniería de Software	Marcela Mederos
Tutoría de Proyecto UTULAB	Marcela Mederos
Ciberseguridad	Nibia Venturino

RESUMEN DEL TRABAJO

En el instituto, las fallas de equipos, los pedidos de preparación de laboratorios y el control de equipos se manejaban con planillas en papel, avisos verbales y correos. No había registro único ni número de seguimiento.

La solución permite que:

- Los **docentes** registren incidencias mediante tickets y soliciten la preparación de laboratorios.

- Los **técnicos** atiendan tickets, registren diagnósticos, reparaciones y soluciones, y gestionen equipos, su historial y sus préstamos.
- La **coordinación** administre usuarios y consulte un panel de métricas.

El acceso se controla por roles. La interfaz está en español e inglés y se adapta a PC, tablet y celular. El trabajo se hizo en tres entregas con modelo incremental. Resultado: sistema funcional, prototipo navegable igual al producto, 52 de 52 casos de prueba aprobados y validación final donde los participantes completaron las tareas, la mayoría sin ayuda.

Palabras clave: mesa de ayuda, gestión de recursos tecnológicos, trazabilidad de equipos, soporte informático, control de acceso por roles, sistema web.

INTRODUCCIÓN Y CONTEXTO

La letra del proyecto (abril 2026) plantea que el equipo es contratado para digitalizar la gestión tecnológica de la institución. La Coordinación de Informática y su equipo de soporte mantienen los laboratorios y equipos que se usan a diario.

El relevamiento mostró tres problemas por sector:

- **Docentes:** reportan las fallas (los estudiantes avisan al profesor), pero luego no saben en qué estado está su pedido.
- **Técnicos:** reciben pedidos por distintos canales, sin prioridades, y pierden tiempo respondiendo consultas sobre el avance.
- **Coordinación:** no tiene visión general del estado de los equipos ni datos para planificar compras o reemplazos.

Antecedentes: ya existe un inventario general que se actualiza al ingresar equipos, y el soporte planea a futuro un sistema de préstamos de laptops.

OBJETIVO GENERAL

Desarrollar e implementar el SGRSI, una plataforma web interna centrada en la gestión de recursos tecnológicos y del soporte informático, que reemplace los registros en papel y los avisos verbales por una herramienta con seguimiento y trazabilidad.

OBJETIVOS ESPECÍFICOS

1. Relevar el funcionamiento actual del soporte con entrevistas y observación.
2. Implementar una mesa de ayuda con tickets (estado y prioridad), diagnósticos, reparaciones y soluciones.
3. Implementar la gestión de equipos con historial técnico, movimientos, préstamos y devoluciones.
4. Crear un canal formal para pedir preparación de laboratorios e instalación de software.
5. Establecer control de acceso por roles (Coordinador, Técnico, Docente) con inicio y cierre de sesión.
6. Desarrollar un panel de métricas para la coordinación.
7. Ofrecer interfaz bilingüe y adaptable a PC, tablet y celular.
8. Validar con pruebas funcionales y pruebas con usuarios.
9. Documentar con manuales de usuario e instalación, y preparar el despliegue en servidor con respaldo.

ALCANCE

El SGRSI cubre: registro y seguimiento de incidencias, trabajo técnico sobre cada una, solicitudes de laboratorio, gestión de equipos con historial y préstamos, administración de usuarios y métricas. Funciona dentro de la red institucional.

- Usuarios: docentes (solicitantes), técnicos y coordinación. Un usuario puede tener varios roles y elegir con cuál entrar.
- Los estudiantes no son usuarios; pueden aparecer en la información de una incidencia, pero el reporte lo hace el docente.
- Las cuentas no se crean por cuenta propia: las registra el coordinador.
- El sistema ordena el trabajo manual del soporte, no lo reemplaza, y no sustituye el inventario general existente.
- Fuera de alcance: pruebas de carga y rendimiento con muchos usuarios simultáneos.

LIMITACIONES

- La calidad de la información depende de que docentes y técnicos registren bien y a tiempo.
- El acceso depende de la red y del servidor del instituto.
- La validación final se hizo con estudiantes que asumieron los roles.
- El desarrollo estuvo condicionado por el tiempo compartido con otras materias y la experiencia previa del equipo.

ESTRUCTURA DEL DOCUMENTO ORIGINAL

Primero se presenta el proyecto (resumen, introducción, objetivos, alcance). Luego cada unidad curricular, en el orden de la letra: UTULAB (proceso de diseño, validación, evolución y trabajo en equipo), Ingeniería de Software (relevamiento, requerimientos, modelos, métricas, costos, testing, manuales), Sistemas Operativos (servidor, contenedores, respaldo), Programación Full Stack (API, datos, seguridad del código) y Ciberseguridad (ataques, prevención y detección). Cierra con conclusiones, reflexiones, glosario, bibliografía y anexos. Un tema compartido se desarrolla en una sola sección y las demás la referencian.

2. TUTORÍA DE PROYECTO UTULAB

UTULAB documenta el proceso de diseño: del diagnóstico inicial a un sistema validado con usuarios, en tres entregas.

2.1 PRIMERA ENTREGA: DIAGNÓSTICO

Se detectaron ineficiencias por registros manuales y planillas dispersas para inventario, préstamos e incidencias. Se propuso una intranet con control de acceso por roles (Solicitante, Técnico, Administrador).

ENTREVISTA AL EQUIPO DE SOPORTE

Objetivo: identificar fallas cotidianas, límites de las herramientas actuales y expectativas. Se hicieron 13 preguntas. Respuestas principales:

Tema	Respuesta obtenida
¿El estudiante es usuario?	No. Avisa al profesor y el docente ingresa el ticket.
Prioridad de tickets	Por gravedad y orden de llegada (ej.: PC que no enciende > falta un mouse). Colores tipo semáforo y fechas límite.
Préstamos y equipos	Registrar n.º de inventario, serie, marca, modelo, estudiante, fecha de entrega y devolución. El inventario general ya existe.
Solicitud de servicio vs. ticket	Son conceptos distintos; el sistema debe diferenciarlos.
Administrador	Necesario; los usuarios no crean sus cuentas.
Estadísticas	Equipos con más problemas, más dañados, periféricos rotos; sirve para planificar compras.
Ticket vencido	Notificación de aviso con tiempo transcurrido; se puede registrar por qué no se reparó.
Aprobación previa	No hace falta. No hay anticipación mínima; se guían por urgencia. Hoy se pide por correo.
Validaciones	Campos obligatorios, sin malas palabras, menú de incidentes comunes (internet, PC, mouse, TV, cables). Registrar laboratorio, docente y grupo.
Copias de seguridad	Cuando sea necesario.
Cierre de ticket	No requiere justificación; basta con indicar que se solucionó.
Varios tickets por PC	Sí, puede haber varios. Registrar uso de docentes, incluso con casilla "equipo no utilizado".

Conclusión: el sistema se orienta al docente como solicitante, centraliza incidencias con fecha, prioridad, estado y fecha límite, genera avisos, gestiona usuarios por un administrador, registra préstamos y produce estadísticas útiles para compras.

OBSERVACIÓN DIRECTA NO PARTICIPANTE

Se observó el flujo de solicitudes docentes durante la jornada. Los pedidos se hacen en papel entregado en mano o en forma verbal, sin registro central ni número de seguimiento. Esto genera demoras, pérdidas de solicitudes y falta de trazabilidad.

USUARIOS Y ACTORES

Rol	Necesidades	Responsabilidades	Situación actual
Solicitante (Docente)	Reportar fallas rápido, pedir equipos o servicios, conocer el estado	Completar bien los datos, usar responsablemente los equipos	Reporta de forma dispersa, sin seguimiento
Técnico	Ver y priorizar tickets, consultar inventario, registrar diagnóstico y solución	Resolver incidencias, actualizar estados, gestionar préstamos	Recibe pedidos dispersos sin prioridades
Administrador / Coordinador	Gestionar usuarios, controlar inventario, supervisar, obtener reportes	Altas, bajas y cambios de usuarios y equipos; decidir con datos	Sin visión centralizada

PROBLEMA CENTRAL

La gestión basada en registros manuales, comunicación verbal y planillas aisladas impide la trazabilidad. Provoca inventario desactualizado, descontrol sobre equipos prestados y saturación de la mesa de ayuda por falta de un canal formal. El análisis de factibilidad se trasladó a Ingeniería de Software (sección 3.3 de este resumen).

FODA DEL EQUIPO

	Positivo	Negativo
Interno	Fortalezas: compromiso, perseverancia, amistad, motivación	Debilidades: actividades extracurriculares, superposición con otras materias, uso no óptimo de Trello
Externo	Oportunidades: conocidos con formación, equipos potentes, consultas con docentes	Amenazas: cortes de conectividad, ausencia de profesores clave

TABLA DE DECISIÓN POR ROL

Rol	Condición	Acción	Estado resultante
Docente	Formulario completo	Crear ticket	Ticket registrado
Docente	Formulario incompleto	Mostrar error	No enviado
Técnico	Ticket sin asignar	Consultar bandeja	Pendiente
Técnico	Asignado, no resuelto	Diagnosticar	En proceso
Técnico	Asignado y diagnosticado	Registrar solución	Resuelto
Coordinador	Gestionar el sistema	Usuarios, reportes, estadísticas	Según módulo

PRIMERA PROPUESTA DE SOLUCIÓN

Una intranet cliente-servidor con cinco módulos:

1. **Gestión de inventario:** alta, modificación y consulta de equipos, con estado y ubicación.
2. **Trazabilidad de préstamos:** posesión y devolución de equipos prestados.
3. **Solicitudes de servicio y mesa de ayuda:** tickets con recepción, prioridad, diagnóstico y resolución.
4. **Usuarios y control de acceso por roles.**
5. **Dashboard de métricas** para decisiones de la coordinación.

Beneficios esperados: eliminar planillas, reducir pérdidas de equipos, mejorar el seguimiento, controlar el inventario y generar métricas. El módulo de préstamos se incluyó aunque el soporte no lo consideraba crítico, porque la letra lo pide y evita reestructurar más adelante.

Ciclo de vida inicial: cascada con prototipado, justificado por requerimientos estables, validación temprana con prototipos y un calendario de entregas secuencial. Etapas: análisis, diseño, construcción, pruebas e integración, despliegue. (Luego se cambió a modelo incremental.)

REPRESENTACIÓN INICIAL Y TAREAS

Plantilla con barra superior (sistema, fecha, usuario y rol), menú lateral que cambia según el rol y contenedor central.

- Vista Solicitante: Inicio, Reportar Incidencia, Mis Tickets, Agendar Laboratorio.
- Vista Técnico: Bandeja de Tickets, Historial de Soporte, Consulta de Inventario.
- Vista Administrador: todo lo anterior más Control de Inventario, Trazabilidad de Préstamos y Gestión de Usuarios.

Tareas frontend: F1 estructura HTML5, F2 estilos CSS3, F3 validación en JavaScript. Tareas backend: B1 modelo y normalización de la base, B2 script SQL con datos semilla, B3 lógica de servidor para autenticación y consultas.

Prueba de concepto: se usó XAMPP para vincular HTML, PHP y SQL. La conexión con la base todavía no era del todo funcional.

ORGANIZACIÓN DEL EQUIPO

Toma de decisiones democrática, con coordinador y subcoordinador como moderadores. Comunicación por WhatsApp (avisos) y Discord (reuniones). Ante incumplimientos, se indicaría con sinceridad en la entrega quién no cumplió.

Reflexión inicial: se priorizaron tecnologías conocidas (HTML, CSS, JS, PHP, SQL). Hubo problemas de organización, superposición con otras materias y ausencia de docentes. Aprendizaje: el compromiso no alcanza sin planificación rigurosa. Mejoras: Trello semanal, reuniones breves de seguimiento y más margen en los plazos. El arranque fue flojo, pero dejó una base sólida.

2.2 SEGUNDA ENTREGA: PROTOTIPO Y VALIDACIÓN

Se profundizó el problema:

- **Falta de visibilidad:** el docente no sabe el estado de su pedido ni cuándo estará listo un laboratorio.
- **Sobrecarga del técnico:** pierde tiempo respondiendo consultas verbales sobre el avance.
- **Predicción de la demanda:** la preparación de laboratorios es operativa para planificar las clases.

Prototipo digital navegable: navegación limpia y minimalista. Docente: inicio con accesos a Reportes y Solicitar Laboratorio, lista de solicitudes con estados por color. Técnico: bandeja con priorización, cambio de estado y diagnóstico. Coordinador: altas, bajas y modificaciones, y dashboard con equipos con más fallas.

Validación: encuesta en Google Forms a docentes y estudiantes (rol, frecuencia de uso de soporte, forma actual de resolver problemas, primeras impresiones, tareas guiadas y evaluación). Intención de uso entre 6 y 10.

Hallazgo	Ajuste decidido
Se valoró la estética, colores y orden visual	Se mantiene la línea visual
No se distinguía el rol activo	Cabecera fija "Usuario: [Nombre] / Rol: [Rol]"
Confusión entre "Reportes" e "Incidencias"	"Nuevo Reporte / Crear Ticket" y tooltip "Consulta e Historial de Reparaciones"
Faltaban vistas de Solicitudes de Servicio	Diseñarlas en Figma
Logos secundarios no armonizaban	Armonizar con la paleta
El Figma difería del producto real	Rediseñar el Figma completo

Merchandising inicial: Deklan Enterprise se presenta como proveedora de software para instituciones y empresas. Logo tecnológico y simple, paleta de azules con dorado (referencia a UTU), Arial en el sistema y Times New Roman en la documentación. Lema: "DeKlan Enterprise: Arquitectos del software que define el mañana".

Se distingue entre usuarios (docentes, técnicos, coordinadores) y quienes deciden contratar (direcciones y responsables de tecnología). Medios: Instagram (vitrina principal), LinkedIn (perfil profesional), correo con demostración presencial, y merchandising físico (tarjetas, remeras polo, stickers para ferias). Plan simulado en Instagram: lanzamiento, video demostrativo y infografía de beneficios.

2.3 TERCERA ENTREGA: CIERRE

Funcionamiento del equipo: se reforzó el tablero Kanban en Trello. Mejoró la distribución del trabajo respecto a la primera entrega; se mantuvieron el compromiso y la amistad; sigue pendiente no concentrar el esfuerzo en los días previos a la entrega.

PROTOTIPO FINAL

Prototipo de alta fidelidad en Figma con **26 pantallas** navegables por rol, con las mismas pantallas, colores, textos y datos de prueba que el sistema real. Empieza en el inicio de sesión; cada formulario muestra el mensaje de confirmación real.

- **Acceso:** login con cédula y contraseña, error ante datos incorrectos, selector de rol.
- **Coordinador:** listado de usuarios con roles y estado; alta y edición (cédula, nombre, apellido, contraseña, roles).
- **Docente:** inicio, ingreso de tickets (espacio, equipo, asunto, descripción, turno, grupo, profesor) y solicitud de laboratorio (laboratorio, software, detalle, restricciones, fecha, hora).
- **Técnico:** panel de métricas, tickets con buscador y filtros, solicitudes de laboratorio, diagnósticos, reparaciones, soluciones, equipos e historial técnico.

Ajustes respecto a la segunda entrega: nombre y rol en la cabecera, menú con nombres de acción, vistas de laboratorio agregadas, Figma igual al producto, y cambio de idioma español/inglés.

VALIDACIÓN FINAL

Realizada el 19 de octubre con seis estudiantes: tres como docente, dos como técnico y uno como coordinador. Tareas guiadas y encuesta breve. Además, los casos de uso

CU01, CU02, CU05, CU14 y CU26 quedaron validados con todos sus casos de prueba aprobados.

Tarea	¿Sin ayuda?	Dificultad observada	Comentario del usuario
Iniciar sesión y elegir rol	Sí	Dos tardaron en entender el selector de rol	Ingreso rápido; la cabecera deja claro el rol
Ingresar ticket	Sí	Duda entre "Asunto" y "Descripción"	Formulario completo; la lista de equipos ayuda
Solicitar laboratorio	2 de 3	No supo qué poner en "Restricciones"	Más práctico que el correo
Diagnóstico y solución	1 de 2	Orden poco claro entre diagnóstico, reparación y solución	Buscador y filtros útiles
Agregar usuario	Sí	Ninguna	Útil asignar varios roles

Sugerencias: textos de ayuda en los campos, explicación en el selector de rol y pasos a seguir en la vista del técnico. Análisis: todas las tareas se completaron; ya no aparecieron los problemas de rol ni de menú de la segunda entrega.

IDEA INICIAL VS. SOLUCIÓN FINAL

Aspecto	Idea inicial	Solución final
Módulos	Préstamos y mesa de ayuda	Usuarios y roles, mesa de ayuda completa, laboratorios, equipos, historial técnico y métricas
Roles	Solicitante, Técnico, Administrador	Docente, Técnico, Coordinador; varios roles por usuario
Navegación	Menú lateral	Barra superior, desplegable en celular
Representación	Descripción textual	Figma de alta fidelidad igual al sistema

Aspecto	Idea inicial	Solución final
Idioma	Español	Español e inglés
Tecnología	Pruebas en XAMPP	PHP con MVC, base relacional y JSON

Motivos de los cambios: la primera devolución marcó que faltaban módulos; los préstamos se postergaron por no ser críticos; "Solicitante" pasó a "Docente" por claridad; la barra superior gana espacio y se adapta mejor a celular; el Figma se rehízo; el registro técnico se dividió en diagnóstico, reparación y solución para formar un historial por equipo.

INFORME REFLEXIVO

Grupal: se aprendió a relevar un problema real, convertirlo en propuesta y validarlo con usuarios, y que el prototipo debe acompañar al producto. Dificultades: arranque con mala organización y primera entrega enfocada en Ingeniería de Software. Estrategias: usar las devoluciones como guía, revisar letra y rúbrica, Kanban, referentes por materia y fechas internas de cierre. La evolución fue notoria.

Individuales:

- **Julián González:** descubrió que se adapta bien a la adversidad y mejoró el manejo de tiempos. Lo volvería a hacer.
- **Santiago Cuello:** aprendió a adaptarse a cambios y a manejar tiempos; llegó tranquilo a la última entrega.
- **Ezequiel Forcelledo:** aprendió a no confiarse con los plazos; armó la estrategia de dedicar un día entero al proyecto.
- **Federico Venis:** integró lo aprendido en las materias y aprendió a justificar decisiones más allá del código.

IMPACTO DEL PROYECTO

Antes	Con el SGRSI
Incidencias en papel o verbales que se perdían	Ticket con número, estado y prioridad

Antes	Con el SGRSI
El docente no conocía el estado	Estados registrados: Pendiente, En Proceso, Resuelto, Cerrado
Sin historial de reparaciones	Diagnóstico, reparación y solución forman el historial por equipo
Laboratorios pedidos por correo o en persona	Solicitud formal con laboratorio, software, fecha y hora
Sin datos para comprar o reemplazar	Métricas por estado, por salón y tiempos de resolución
Acceso libre a la información	Usuario, contraseña y permisos por rol

COMUNICACIÓN Y MERCHANDISING FINAL

- **Identidad:** azules (tecnología e institución) con un naranja llamativo; Arial y Times New Roman; mismo logo y colores en sistema, prototipo, documentación y difusión.
- **Producto:** web que centraliza el soporte con tickets, diagnósticos, reparaciones, laboratorios, equipos y métricas; dirigido a instituciones educativas y empresas.
- **Diferencial:** diseñado a partir de una reunión con un equipo de soporte real, pantallas por rol, bilingüe, responsivo e historial técnico por equipo.
- **Piezas visuales:** login y dashboard del coordinador en PC y celular.
- **Estrategia final:** se dirige a direcciones y coordinaciones vía LinkedIn, correo y demostraciones, con tarjetas y folleto; Instagram como vitrina; stickers para ferias.

3. INGENIERÍA DE SOFTWARE

Esta área reúne la documentación técnica de las tres entregas: relevamiento, factibilidad y requerimientos (1.ª), modelo, casos de uso, UML y riesgos (2.ª), y costos, métricas, testing y manuales (final).

3.1 RELEVAMIENTO

Técnica: entrevista semiestructurada a los futuros usuarios. Resultados:

- Los docentes registran las incidencias; los estudiantes avisan al docente.
- Los técnicos atienden y resuelven; el coordinador administra usuarios y supervisa.
- Las incidencias se gestionan con tickets y se guarda historial de incidencias, reparaciones e intervenciones.
- Las solicitudes de servicio son un proceso aparte.
- Se necesitan estadísticas e indicadores.
- Datos obligatorios: laboratorio, docente, grupo y descripción.
- Seguimiento por estados y fechas de finalización.

3.2 FACTIBILIDAD

Tipo	Conclusión
Técnica	Viable con HTML, CSS, JavaScript y un gestor de base de datos conocidos; la institución tiene equipos y conectividad.
Operativa	Responde a una necesidad real; reemplaza el papel por un registro digital central.
Económica	Alta: herramientas gratuitas; reduce papel y tiempo. Costos limitados al tiempo de desarrollo y mantenimiento.
Recursos humanos	El equipo conoce programación web, bases de datos y documentación; se requiere buena organización.
Temporal	Factible dentro de los plazos, con seguimiento continuo ante cambios.

Conclusión: el SGRSI es factible.

3.3 REQUERIMIENTOS FUNCIONALES (61)

Grupo	RF	Qué cubre
Catalogación de activos	RF1–RF10	Registrar equipos, marca, estado, ID interno, ubicación; modificar, consultar, buscar por serie y filtrar por estado o ubicación
Trazabilidad	RF11–RF14	Registrar cambios de estado y ubicación; consultar movimientos e historial de estados
Disponibilidad en tiempo real	RF15–RF18	Mostrar disponibles, fuera de servicio y estado actual
Registro de incidencias	RF19–RF30	Crear tickets asociados a equipo y laboratorio, con descripción, fecha, turno, taller, docente y grupo; consultar por equipo; filtrar por prioridad
Seguimiento de tickets	RF31–RF39	Ver pendientes, en proceso y resueltos; cambiar estado; fecha de finalización; buscar por palabra; filtrar por estado y fechas; ver ID
Diagnósticos	RF40–RF45	Registrar, asociar, consultar y modificar diagnósticos y soluciones
Historial técnico	RF46–RF49	Reparaciones, intervenciones y reemplazos por equipo
Autenticación	RF50–RF51	Iniciar y cerrar sesión
Dashboard	RF52–RF55	Estadísticas de incidencias, resueltas, tiempos de resolución y uso de laboratorios
Solicitudes de servicio	RF56–RF61	Preparación de laboratorios, instalación de software con versión, restricciones para estudiantes, consulta y filtro por tipo

Cada grupo define alcance y limitaciones. La limitación común: la calidad depende de que los usuarios registren y actualicen bien los datos.

3.4 REQUERIMIENTOS NO FUNCIONALES (28)

Categoría	RNF	Exigencia
Plataforma web	1–2	Web dentro de la intranet
Seguridad	3–5	Usuario y contraseña, acceso por rol, bloqueo a no autenticados
Disponibilidad	6	Acceso simultáneo en horario institucional
Rendimiento	7–8	Consultas y registros en menos de 5 segundos
Usabilidad	9–10	Máximo 5 clics; menús uniformes
Escalabilidad	11–12	Agregar módulos sin tocar los existentes
Mantenibilidad	13–15	Código comentado, modular y documentación actualizada
Compatibilidad	16	Última versión de Chrome
Idiomas	17–18	Español e inglés
Integridad	19–21	Consistencia, sin campos obligatorios vacíos, sin duplicados críticos
Persistencia	22	Base de datos relacional
Responsividad	23–25	Celular hasta 576 px, tablet hasta 768 px, monitor desde 1024 px
Manuales	26–28	Manual de instalación, configuración del entorno y manual de usuario

3.5 FODA DEL SISTEMA

- **Fortalezas (impacto alto salvo indicación):** digitaliza hoja de laboratorios y registro de equipos; historial de reparaciones; control de incidencias; historial por equipo; interfaz intuitiva (medio); tickets centralizados; menos pérdida de información; menos papel (medio).
- **Oportunidades:** sugerencias de usuarios (medio); necesidad de digitalizar; mayor importancia del control de equipos; necesidad de historial organizado.

- **Debilidades:** dependencia del buen registro (alto); capacitación inicial (medio); errores de carga (medio); dependencia de la red (alto); mantenimiento constante (medio).
- **Amenazas:** accesos no autorizados, pérdida por fallas del servidor, fallos de respaldo y ataques (alto); borrado accidental y obsolescencia tecnológica (medio).

3.6 PLANIFICACIÓN Y MODELO DE DESARROLLO

Referentes: Santiago (Ingeniería de Software), Federico (Programación), Julián (UTULAB), Ezequiel (Sistemas Operativos), con participación de todos en todo. Organización colaborativa, tablero Kanban, requerimientos como Issues de GitHub. Reglamento: tareas equitativas, plazos, avisar cambios, actas, reuniones obligatorias (justificar ausencias). Comunicación por Google Meet y WhatsApp; reuniones mayormente virtuales, aceptado por la docente. Se elaboró un diagrama PERT.

Cambio de modelo: el híbrido cascada-ágil inicial no reflejaba la práctica. Se adoptó el **modelo incremental**, donde cada entrega suma una parte funcional y se corrige con las devoluciones.

Incremento	Fecha	Contenido
1	Junio	Relevamiento, requerimientos, factibilidad, DER, prototipo Figma, maquetación inicial
2	Agosto	Modelo esencial, casos de uso, diagrama de clases, riesgos, backend PHP con base, login y roles
3	Octubre	API REST, CRUD completo, pruebas, manuales y cierre

Justificación: coincide con las entregas parciales, permite validar temprano, incorpora devoluciones sin rehacer todo y reduce riesgos.

Modelo ambiental: impacto mayormente positivo (menos papel, tinta y archivo físico), compensado con optimización de almacenamiento y compresión de imágenes. También se elaboró un modelo de comportamiento.

3.7 CASOS DE USO

Cada caso tiene diagrama y ficha desplegada (actor, inclusiones, pre y postcondiciones, flujo principal y alternativos).

Caso de uso	Actor	Flujo principal resumido	Alternativos
Iniciar sesión	Todos	Ingresa usuario y contraseña; el sistema valida y abre según el rol	Credenciales incorrectas; usuario inactivo
Cerrar sesión	Todos	Finaliza la sesión, borra datos y vuelve al login	Ninguno
Modificar usuario	Coordinador	Edita contraseña y/o rol; valida y guarda	Datos inválidos. La CI no se puede cambiar
Registrar usuario	Técnico/Asistente*	Ingresa CI, contraseña y rol; valida y guarda	Datos inválidos; CI ya existente
Eliminar usuario	Coordinador	Selecciona, confirma y elimina	Cancela
Limpiar filtros de usuarios	Coordinador	Restablece el filtro de rol y muestra todos	Sin filtros aplicados
Registrar equipo	Técnico	Elige ID, laboratorio, marca, estado, disponibilidad e información	Datos inválidos; ID repetido
Filtrar equipo por ID	Técnico	Ingresa ID y ve solo ese equipo	Sin resultados; campo vacío muestra todos
Filtrar por laboratorio /	Técnico	Elige el valor y ve los equipos que coinciden	Sin resultados; restablecer filtro

Caso de uso	Actor	Flujo principal resumido	Alternativos
estado / disponibilidad			
Modificar equipo	Técnico	Cambia marca, laboratorio, estado o disponibilidad y guarda	Datos inválidos; cancela
Eliminar equipo	Técnico	Confirma y elimina	Cancela
Buscar equipo por ID	Técnico	Ingresa ID o serie y ve el equipo	No encontrado; campo vacío
Registrar ticket	Docente	Laboratorio, equipo, asunto, descripción, turno, grupo y docente; queda Pendiente	Falta un dato obligatorio
Buscar ticket	Técnico	Palabra clave o ID	Sin resultados; borrar criterio
Filtrar ticket por prioridad / estado	Técnico	Alta, Media o Baja; o el estado elegido	Sin resultados; restablecer
Filtrar ticket por fechas	Técnico	Rango desde-hasta, validando el orden	Rango inválido; sin resultados
Cambiar estado del ticket	Técnico	Elige el nuevo estado; si es Resuelto, guarda fecha de finalización	Ninguno
Filtrar incidencias por equipo	Técnico	Elige equipo y ve sus incidencias	Sin incidencias; quitar filtro
Registrar diagnóstico	Técnico	Elige incidencia, escribe y guarda	Descripción vacía

Caso de uso	Actor	Flujo principal resumido	Alternativos
Registrar solución	Técnico	Elige diagnóstico, escribe y guarda	Descripción vacía
Modificar diagnóstico	Técnico	Edita y guarda	Vacío; cancela
Consultar historial técnico	Técnico	Elige equipo y ve sus registros con detalle	Sin historial
Registrar solicitud de laboratorio	Docente	Laboratorio, si requiere software, requerimientos, restricciones, fecha y hora	Pide software (nombre y versión); falta dato

*En el documento original, el caso "Registrar usuario" figura con actor Técnico/Asistente, aunque en el resto del sistema el alta la hace el Coordinador.

3.8 DIAGRAMAS, RIESGOS Y FODA PONDERADO

Diagrama de clases final: páginas HTML que llaman por JavaScript a un único punto de entrada (public/index.php), que deriva cada ruta a su controlador; el controlador valida y usa su DAO, que ejecuta consultas preparadas. También hay diagramas de estados para tickets y equipos.

Riesgo	Probabilidad	Impacto	Contingencia
Retraso en tareas	Media	Alto	Redistribuir y priorizar
Ausencia de un integrante	Media	Medio	Repartir tareas usando documentación y repositorio
Pérdida de código	Baja	Alto	Versiones en GitHub
Conflictos al integrar	Alta	Medio	Ramas y revisión antes de integrar
Errores en la base	Media-Alta	Alto	Conservar DDL y DML para reconstruir

Riesgo	Probabilidad	Impacto	Contingencia
Falla del servidor	Baja	Alto	Mecanismos de recuperación
Funcionalidades con errores	Media	Alto	Pruebas y corrección antes de cada entrega
Cambios o módulos nuevos	Media	Medio	Analizar impacto y organizar tareas
Mala comunicación	Media-Alta	Medio	Reuniones y registro de decisiones
Incumplir una entrega	Baja	Alto	Priorizar y revisar avance antes

FODA ponderado: cada factor se califica de 1 a 5 y se pondera por peso (internos suman 100 %, externos 100 %).

Comparación	Puntajes	Diferencia
Fortalezas vs. debilidades	2,60 vs. 1,40	+1,20
Oportunidades vs. amenazas	1,81 vs. 1,68	+0,13

Conviene aprovechar las fortalezas (digitalización, historial, dashboard). La diferencia externa es pequeña: los accesos no autorizados y la pérdida de datos se mitigan con roles, contraseñas cifradas y servidor de respaldo.

3.9 MÉTRICAS Y COSTO-BENEFICIO

Se usó el método de **Puntos de Función** (entradas, salidas, consultas, archivos e interfaces, con ajuste de complejidad).

Métrica	Valor
Puntos de Función	202,8 PF
LDC estimadas (PF × 30)	6.084
Valor esperado PERT	6.164,7 LDC
LDC medición anterior	4.743

Métrica	Valor
LDC reales finales (PHP y JS)	5.159

El sistema quedó 925 líneas por debajo de lo estimado gracias a la reutilización: un único index.php, clases comunes de validación y conexión, y un núcleo JS compartido. Diferencias: las LDC miden tamaño físico y dependen del lenguaje (en Ensamblador serían 64.896); los PF miden funcionalidad, no dependen del lenguaje y se calculan desde el análisis. Conclusión: PF sirvió para estimar desde el inicio y LDC para validar con el producto real.

Costo-beneficio (5 años, US$ 12 la hora, 40 semanas lectivas, software libre):

Concepto	Monto (US$)
Desarrollo	23.863
Disco del servidor de respaldo	300
Capacitación (10 h)	120
Costo inicial total	**24.283**
Mantenimiento anual (10 %)	2.386
Ahorro coordinación (400 h/año)	4.800
Ahorro técnicos (200 h/año)	2.400
Menos pérdidas de equipos (2 × 500)	1.000
Ahorro en papel	200
Beneficio anual	**8.400**
Beneficio neto anual	6.014
Beneficios a 5 años	42.000
Costos a 5 años	36.213
Ganancia neta a 5 años	**5.787**

Recuperación de la inversión: 4,04 años. Relación beneficio/costo: 1,16, por lo que el proyecto es económicamente aceptable. Beneficios intangibles: decisiones con datos, respuesta más rápida, laboratorios a tiempo, mejor organización e historial por equipo.

3.10 TESTING

Las pruebas se hicieron sobre la versión final con **Playwright**, que automatizó los 52 casos y permitió repetirlos tras cada cambio. Entorno: XAMPP local, Chrome y base con datos de prueba. Responsable registrado: Augusto Cuello (los cuatro módulos).

- **Objetivo:** verificar los requerimientos con datos válidos e inválidos y el acceso por rol.
- **Módulos:** Solicitante, Técnico, Administrador y Métricas.
- **Tipos:** caja negra, positivos y negativos, por rol, validación de casos de uso, tablas de decisión y regresión.
- **Errores:** crítico (impide una función o pierde datos), mayor (resultado incorrecto), menor (visual o de texto).
- **Aprobación:** 90 % o más de casos aprobados y sin críticos pendientes.
- **Cronograma:** 13–14/10 preparación, 15–17/10 ejecución, 18–19/10 correcciones, 20/10 regresión, 21/10 cierre.
- **Método:** Playwright crea usuarios por rol y sus datos; en los negativos también consulta la base para confirmar que no se guardó nada.

Resultado: 52 ejecutados, 52 aprobados, 0 fallidos, 100 %, unos 42 segundos.

Sector	Casos	Positivos	Negativos	Qué se comprobó
Autenticación	4	1	3	Login correcto; contraseña errónea; CI inexistente con el mismo mensaje; campos vacíos
Usuarios y roles	7	6	1	Alta con rol Coordinador; paneles Técnico y Docente; selector con varios roles; inactivo bloqueado; editar; desactivar

Sector	Casos	Positivos	Negativos	Qué se comprobó
Tickets	11	7	4	Alta con número; formulario vacío; asunto con espacios rechazado; sin conexión; error 500 controlado; listado; filtros por prioridad, estado, fechas y equipo; cambio de estado
Diagnósticos	5	4	1	Registrar; texto entre 10 y 2000 caracteres; consultar; consultar desde ticket; modificar
Soluciones	2	1	1	Registrar; vacía rechazada
Reparaciones	2	1	1	Registrar; vacía rechazada
Historial técnico	2	2	0	Equipo sin reparaciones con aviso; reparación visible en historial
Equipos	6	5	1	Listar; filtros; pasar a En mantenimiento; eliminar; ID repetido rechazado; filtrar Dañado
Laboratorios	3	3	0	Ver solicitudes; filtrar por fecha y limpiar; ingresar solicitud
Idioma	1	1	0	EN se mantiene al cambiar de página
Control de acceso	3	1	2	Sin sesión vuelve al login; rol sin permiso bloqueado; cerrar sesión
Préstamos	2	2	0	Préstamo deja el equipo No disponible; devolución lo libera
Dashboard	1	1	0	Contadores coinciden con la base
Intervenciones y reemplazos	3	2	1	Intervención; descripción corta rechazada; reemplazo
Total	**52**	**37**	**15**	

Validación de casos de uso con casos de prueba:

Caso de uso	Módulo	Casos de prueba	Resultado
CU05 Ingresar usuario	Administrador	T-05	Validado
CU01 Inicio de sesión	Solicitante	T-02, T-03, T-04, T-07, T-09, T-45	Validado
CU14 Registrar ticket	Solicitante	T-12 a T-16	Validado
CU26 Preparación de laboratorio	Solicitante	T-42	Validado
CU02 Cerrar sesión	Solicitante	T-44, T-46	Validado

Cada ficha detalla flujos alternativos. Ejemplos: en el alta de usuario se valida cédula de 8 dígitos, nombre (1–12) y apellido (1–16), contraseña, al menos un rol y cédula no repetida; en la solicitud de laboratorio se valida que exista el laboratorio, que haya detalle si se pide software y que fecha y hora tengan formato válido.

3.11 TABLAS DE DECISIÓN

Cinco tablas modelan las validaciones en el orden en que el sistema las evalúa:

1. **Inicio de sesión (CU01):** campos completos → cédula registrada → usuario activo → contraseña correcta → tiene rol → más de un rol. Acciones: marcar campos, "credenciales incorrectas", "usuario inactivo", "sin roles", abrir panel o abrir el selector de rol.
2. **Alta de usuario (CU05):** cédula completa → 8 dígitos → nombre y apellido válidos → contraseña → al menos un rol → cédula no registrada → crear usuario activo.
3. **Registro de ticket (CU14):** campos completos → servidor responde → datos válidos → guardado sin error interno → confirmar.
4. **Preparación de laboratorio (CU26):** campos completos → laboratorio existe → detalle si pide software → formato de fecha y hora → guardar.
5. **Acceso a una página:** hay sesión → rol permitido → mostrar página; si no, volver al login con el aviso correspondiente.

3.12 MANUAL DE USUARIO

Requisitos: dispositivo en la red del instituto, navegador actualizado (Chrome, Firefox o Edge) y credenciales dadas por el Coordinador.

Ingreso: cédula sin puntos ni guiones y contraseña, luego Ingresar. Con varios roles aparece el Selector de rol. Los botones ES/EN cambian el idioma en todo el sistema. "Cerrar sesión" vuelve al login; se recomienda en equipos compartidos.

Coordinador: ve la tabla de usuarios (CI, nombre, apellido, roles, estado). Agregar usuario: CI de 8 dígitos, nombre, apellido, contraseña y roles. Editar: la contraseña solo cambia si se escribe una nueva. Desactivar/Activar en lugar de borrar, para conservar el historial. Dashboard: reportes, incidencias por estado, tiempos y por salón.

Docente: Ingresar Ticket (laboratorio, equipo, asunto, descripción, turno, grupo, profesor) → recibe un número tipo INC-2026-0012 y queda Pendiente. Solicitar laboratorio: espacio, si requiere software (con programa y versión), restricciones, fecha y hora → Publicar Solicitud.

Técnico: Tickets (buscar por número, filtrar por equipo, prioridad, estado o fechas; cambiar estado y prioridad, se guarda solo). Registrar, consultar y modificar diagnósticos (10 a 2000 caracteres). Registrar soluciones y reparaciones sobre un diagnóstico. Equipos: alta, modificación (cada cambio queda en el historial) y eliminación. Préstamos: equipo disponible, cédula y fecha estimada; el equipo pasa a No disponible hasta la devolución. Historial técnico con cambios de estado, movimientos, reparaciones, intervenciones y reemplazos. Solicitudes de laboratorio con filtros por fecha y laboratorio.

3.13 MANUAL DE INSTALACIÓN Y MANTENIMIENTO

Requisitos: Apache con PHP 8+ y PDO MySQL, MySQL o MariaDB, Composer y Git (XAMPP sirve para pruebas locales).

Instalación:

1. Clonar el repositorio de GitHub.
2. Copiar la carpeta PROYECTO al servidor web (htdocs en XAMPP).
3. Ejecutar composer install (instala vlucas/phpdotenv).

4. Crear la base y un usuario con permisos.
5. Ejecutar los scripts DDL en orden: usuario, laboratorio, equipo, ticket, diagnóstico/solución, reparación, préstamo y trazabilidad.
6. Cargar los DML: usuarios, laboratorios, equipos y tickets.
7. Copiar .env.example a .env y completar BD_HOST, BD_USER, BD_PASS y BD_NAME (no se sube al repositorio).
8. Iniciar Apache y MySQL y abrir /PROYECTO/public/.
9. Entrar con el Coordinador inicial, crear usuarios reales y cambiar las contraseñas de prueba.

Verificación: login con cada rol, ticket de prueba visible para el técnico y cambio de idioma.

Mantenimiento: respaldos periódicos; para actualizar, respaldar, git pull, composer install si cambió algo, scripts SQL nuevos y pruebas automatizadas. Los errores se registran con la clase RegistradorErrores en el log de PHP y el usuario solo ve un mensaje controlado. Desactivar usuarios en vez de borrarlos, no compartir el .env y cambiar periódicamente las contraseñas de los coordinadores.

4. ADMINISTRACIÓN DE SISTEMAS OPERATIVOS

Se montó un servidor Ubuntu Server 26.04 LTS físico y, para la entrega final, dos máquinas virtuales Rocky Linux 9.8 (servidor principal y de respaldo), con despliegue en contenedores y respaldo automático en otro equipo.

4.1 INSTALACIÓN DE UBUNTU SERVER

Requisitos: equipo x86-64 con UEFI, pendrive de 8 GB o más, ISO oficial, equipo para Rufus, red, y usuario, contraseña y hostname definidos.

Medio de instalación con Rufus: seleccionar el pendrive y la ISO, esquema GPT, destino UEFI (no CSM), FAT32; presionar Empezar hasta ver "LISTO". Luego configurar el arranque desde USB en el BIOS/UEFI.

Pasos del instalador:

1. Idioma Español.
2. Distribución de teclado (manual o "Identificar teclado").

3. Tipo "Ubuntu Server" estándar, sin drivers de terceros.
4. Red: detecta la interfaz inalámbrica wlp1s0; se elige la red Wi-Fi y su contraseña.
5. Proxy en blanco y repositorio oficial por defecto.
6. Almacenamiento: disco entero con LVM, sin cifrado LUKS. Resultado: partición EFI, partición de arranque y volumen LVM.
7. Perfil: nombre, hostname, usuario y contraseña.
8. Omitir Ubuntu Pro (se activa luego con pro attach).
9. OpenSSH: no se marcó en el principal (se instaló luego), sí en el de respaldo.
10. Instalación, "Installation complete!", retirar el pendrive y reiniciar.

Verificación: login, ip a y ping 8.8.8.8, apt update y upgrade, instalar SSH si falta, hostnamectl, lsblk y vgdisplay.

4.2 ROCKY LINUX EN VIRTUALBOX

Se crearon dos máquinas Rocky Linux 9.8 (la distribución del servidor del ITI) con adaptador de red en modo puente:

Máquina	Hostname	IP
Servidor principal	rocky-server	192.168.1.110
Servidor de respaldo	fede-respaldo	192.168.1.111

Se configuraron con nmcli (método manual, gateway 192.168.1.1, DNS 8.8.8.8) y hostnamectl. Se verificó con os-release, ip a, dnf update y el estado de sshd. El firewall del principal permite SSH, Cockpit y 8080/tcp.

Soporte técnico en tres niveles:

1. **Atención al usuario:** consultas de uso, contraseñas olvidadas, servicios inaccesibles.
2. **Administración del servidor:** acceso por SSH, systemctl, logs en /var/log, reinicio de servicios, restauración desde respaldos.
3. **Soporte externo:** documentación y comunidad de Ubuntu, Ubuntu Pro o soporte comercial de Canonical.

Cada incidencia se registra, clasifica, diagnostica, soluciona y cierra con el usuario. Objetivo: servicio caído resuelto el mismo día hábil y consultas comunes en dos días hábiles.

4.3 ELECCIÓN DEL SISTEMA OPERATIVO

Se eligió Ubuntu Server 26.04 LTS (kernel 7.0.0-38-generic, host fedes-server) por soporte LTS de 5 años, ausencia de entorno gráfico (más recursos libres), integración con Docker y apt, y costo cero.

Sistema	Licencia	Soporte	Paquetes
Ubuntu Server 26.04 LTS	Gratuita	5 años, hasta 10 con Ubuntu Pro	apt
Debian 13	Gratuita	5 años con LTS; extendido por terceros	apt
Rocky Linux 10	Gratuita	10 años	dnf
Windows Server 2025	Paga por núcleos y usuarios	Según fabricante	Windows Update

Ubuntu pide desde 1,5 GB de RAM y 4 GB de disco. Windows contradice el criterio de costo nulo. El script final funciona también en Rocky Linux, detectando firewalld o UFW.

4.4 RED, SSH Y PAQUETES

IP estática: Ubuntu Server usa netplan; se instaló NetworkManager y se delegó la interfaz para usar nmtui. Quedó 192.168.1.100/24 por wlp1s0 con gateway 192.168.1.1. Se probó ping en ambos sentidos con una PC Windows.

Puerto	Servicio	Uso
22/tcp	SSH	Administración y envío de respaldos
80/tcp	Apache	Entorno LAMP
8080/tcp	Contenedor sgrsi-web	Aplicación SGRSI

El puerto 3306 no se publica: MariaDB escucha solo en 127.0.0.1 y la del contenedor está en una red interna.

SSH: instalación, arranque automático y regla de firewall. Conexión desde Windows con ssh fede@192.168.1.100; se creó un archivo de prueba remoto para demostrar el acceso.

Gestión de paquetes con Nginx: se instaló con apt; no arrancó porque XAMPP ocupaba el puerto 80, que se liberó. Se habilitó HTTP en UFW y curl devolvió "Welcome to nginx!".

4.5 ENTORNO LAMP

1. Actualización del sistema (86 paquetes y nuevo kernel).
2. **Apache:** falló por conflicto del puerto 80 con Nginx; se detuvo y deshabilitó Nginx, se habilitó Apache y su regla de firewall. Se verificó la página de bienvenida desde la PC.
3. **MariaDB 11.8.6:** activo, habilitado y escuchando solo en 127.0.0.1:3306.
4. **PHP 8.5.4:** con libapache2-mod-php y php-mysql.
5. **Integración:** archivo info.php con phpinfo() visible desde el navegador.
6. **Estado:** SSH, Apache2 y MariaDB active (running) y enabled. El aviso AH00558 de Apache no afecta el funcionamiento.

4.6 SEGURIDAD Y RESPALDOS

SSH con clave pública: par ed25519 generado en Windows, clave copiada a authorized_keys. Luego se fijó PermitRootLogin no y PasswordAuthentication no.

Primer script de backup: volcaba todas las bases y comprimía /var/www/html; cron diario a las 3:00. Problemas: sin registro, sin limpieza, sin verificación, archivos legibles por todos y uso de sudo en una tarea sin intervención.

Script mejorado (como root):

- umask 077 para que solo root lea los archivos.
- Por ejecución genera cuatro archivos: volcado comprimido de las bases, sitio web, configuración (apache2, ssh, netplan, mysql) y sumas SHA-256.
- Registro en backup.log.

- Retención de 7 días en ~/backups y copia con rsync a /srv/respaldo_secundario durante 30 días.
- Directorio con permisos 700 y archivos 600.
- Se pasó al crontab de root; se probó cada minuto y luego quedó diario a las 03:00, más el respaldo íntegro (30 3 diario y domingo a las 4:00 completo). Zona horaria America/Montevideo.

Pruebas de respaldo y restauración:

Prueba	Método	Resultado esperado
Ejecución manual	Correr el script y ver el log	Registro completo y 4 archivos
Lectura	gzip -t y tar -tzf	Se abren los tres
Archivo dañado	Alterar una copia y sha256sum -c	OK antes, FAILED después
Restaurar base	Borrar la base de prueba y cargar el volcado	Vuelve "dato de prueba"
Restaurar sitio y configuración	Extraer en /tmp y diff -r	Sin diferencias
Copia secundaria	sha256sum -c	Sumas OK
Retención	Archivos con fecha de 10 días	Se borran
Permisos	Leer como nobody	Permiso denegado
Error controlado	Bloquear la carpeta con chattr +i	"ERROR en copia secundaria"
Cron	Programación cada minuto	Ejecuciones minuto a minuto

Alta disponibilidad propuesta (no implementada): RAID 1, snapshots LVM, replicación de MariaDB, segundo servidor web con IP virtual (keepalived), cifrado GPG y avisos por correo. Con respaldo diario, la pérdida máxima es de 24 horas.

4.7 SCRIPT FINAL DEL SERVIDOR

Reúne los cinco scripts de clase en un menú con siete módulos. No usa sudo ni modifica el sistema, por las limitaciones del servidor del ITI. Se probó en Rocky Linux 9.8 con el usuario sin privilegios alumno.

Estructura: admin_servidor.sh (menú principal), config.conf, lib/comunes.sh y la carpeta modulos/ con usuarios, grupos, redes, basedatos, firewall, logs y respaldos. Si falta un módulo, avisa y el resto funciona. Las acciones importantes se anotan en ~/admin_servidor.log. Funciones comunes: titulo, pausa, existe_comando, registrar y nombre_valido.

Módulo	Opciones principales	Comandos
Usuarios	Usuario actual, verificar usuario, todas las cuentas, cuentas de personas (UID ≥ 1000), sesiones abiertas	whoami, id, getent, who
Grupos	Grupos propios, verificar grupo, listar, grupos de un usuario, miembros	id -nG, getent group
Redes	Interfaces, IP, rutas, ruta por defecto, DNS, puertos, conectividad, resolución	ip, resolvectl, ss, ping
Bases de datos	Conexión, permisos, bases, tablas, estructura, SELECT, INSERT/UPDATE/DELETE con confirmación, respaldo	mariadb, mariadb-dump
Firewall	Estado, reglas, zonas, puerto en escucha	firewall-cmd o UFW, ss
Logs	Últimas entradas, por servicio, errores, del usuario, inicios de sesión, búsqueda, log del script	journalctl, last, grep
Respaldos	Crear carpetas, copia con suma, listar, verificar, restaurar, retención de 5, envío al servidor de respaldo	tar, sha256sum, rsync

En bases de datos la contraseña se pide oculta, se pasa por MYSQL_PWD solo durante cada comando y se borra al salir.

Ejecución: copiar con scp, quitar finales de línea de Windows con sed, dar permiso de ejecución y correr ./admin_servidor.sh.

Pruebas destacadas: consulta de red sin sudo con IP 192.168.1.110; "SELECT 1; DROP TABLE" rechazado; DROP en la opción de modificación rechazado; INSERT confirmado; firewalld sin permisos con aviso; puerto 99999 rechazado; retención de 5 copias; copia dañada marcada DAÑADO y no restaurada; restauración idéntica con diff -r.

4.8 CONTENEDORES DOCKER

La aplicación se despliega con docker-compose en dos contenedores:

Elemento	Configuración	Motivo
sgrsi-bd	mariadb:11.8 con variables MARIADB_*	Versión estable fija
Volumen datos_bd	/var/lib/mysql	Los datos sobreviven al recrear
Carga inicial	SQL numerados en docker-entrypoint-initdb.d	Tablas en orden de claves foráneas, solo la primera vez
sgrsi-web	php:8.4-apache con pdo_mysql, raíz en public	La aplicación usa PDO
Contraseñas	.env de despliegue; iniciar.sh crea el .env de la app	No quedan en la imagen ni en el repositorio
red_interna	internal: true (web y bd)	La base no tiene acceso exterior
red_publica	Solo web	Única entrada
Puerto	8080 del servidor al 80	Deja libre el 80
Healthcheck	bd sano antes de iniciar web	Orden correcto de arranque
Reinicio	unless-stopped	Vuelven solos tras reiniciar

Instalación en Rocky: quitar Podman y Buildah, agregar el repositorio de Docker, instalar docker-ce y el plugin compose, habilitar el servicio, agregar el usuario al grupo docker, preparar el .env con permisos 600, abrir 8080/tcp y ejecutar docker compose up -d --build.

Verificación: ambos contenedores healthy, login accesible en 192.168.1.110:8080, 16 tablas creadas, red interna con Internal true, la base sin puertos publicados y volumen sgrsi_datos_bd presente.

gestion_contenedores.sh: menú con 10 opciones: estado, construir/actualizar, detener, reiniciar web o bd, registros, uso de CPU y memoria, volumen y redes, respaldo de la base (conserva 7), restauración con confirmación y eliminar contenedores conservando el volumen. La contraseña de root de MariaDB se usa solo dentro del contenedor.

4.9 SERVIDOR DE RESPALDO

Completa la regla 3-2-1: el servidor principal (192.168.1.110) envía todos los días un respaldo íntegro por SSH al de respaldo (192.168.1.111), y una copia completa del sistema cada semana.

Archivo	Contenido	Frecuencia	Retención
bd_servidor	Bases de MariaDB del servidor	Diaria 03:30	30 días
bd_contenedor	Base del SGRSI en Docker	Diaria 03:30	30 días
archivos	/etc, /home, /root, /var/www y cron	Diaria 03:30	30 días
paquetes	Lista de paquetes instalados	Diaria 03:30	30 días
sistema	Todo el sistema de archivos	Domingo 04:00	Últimas 4
sumas	SHA-256 de cada ejecución	Cada ejecución	Mientras existan sus archivos

Configuración:

1. Rocky Linux mínimo en modo puente, usuario fede-respaldo.
2. Scripts crear_servidor_respaldo.sh y preparar_servidor_principal.sh automatizan todo.
3. IP fija con nmcli y conectividad probada.
4. Carpeta /srv/respaldos en XFS (17 GB en la prueba; en producción conviene un disco dedicado). Una copia completa pesa 3,66 GB.
5. Usuario respaldo sin contraseña, dueño de la carpeta con permisos 700.
6. Clave ed25519 dedicada para root en el principal.

7. La clave está restringida con rrsync -no-del: solo permite rsync en esa carpeta, sin borrar y sin terminal.
8. Firewall que permite SSH solo desde 192.168.1.0/24.

respaldo_integral.sh (principal): se ejecuta como root con umask 077; modo diario (bases, configuración, datos, paquetes) o completo (tar de todo el sistema excluyendo carpetas virtuales, temporales, Docker y respaldos); calcula sumas, envía con rsync, conserva 3 días localmente y registra en /var/log/respaldo_integral.log (código 1 ante error).

verificar_respaldos.sh (respaldo): diario a las 06:00; conserva las 4 copias completas, borra lo de más de 30 días, comprueba cada suma y registra el espacio libre.

Plan de contingencia:

- **Pérdida de datos con el servidor activo:** traer los archivos con la misma clave (rrsync permite leer), verificar la suma y restaurar solo lo necesario.
- **Pérdida del equipo:** reinstalar Rocky con la IP .110, traer la última copia, reinstalar paquetes desde la lista, extraer archivos y aplicar restorecon, restaurar la base, desplegar contenedores y restaurar su base, y comprobar servicios.

Pruebas:

Prueba	Resultado obtenido
Respaldo diario manual	Código 0 y "Envío de respaldos integrales OK"
Archivos recibidos	Dueño respaldo, permisos 600
Abrir terminal con la clave	Rechazado por rrsync
Escribir fuera de la carpeta	Rechazado ("do not use .. in arg")
rsync --delete	Rechazado ("--delete has been disabled")
Verificación en destino	Sumas OK, 0 errores
Archivo modificado en destino	Marcado ERROR; al reenviarlo volvió a OK
Restaurar base del contenedor	Tabla de 16 filas → 0 → 16 tras restaurar

Prueba	Resultado obtenido
Restaurar configuración	sshd_config idéntico al original
Copia completa semanal	3,66 GB recibidos con suma OK

Mejora futura: cifrar con GPG y agregar una copia fuera de la institución.

5. PROGRAMACIÓN FULL STACK

El SGRSI es una aplicación PHP con arquitectura MVC, base MariaDB y una API JSON única consumida con fetch.

5.1 HERRAMIENTAS

Herramienta	Uso	Motivo
Visual Studio Code 1.140.0	Escribir PHP, JS, HTML, CSS y SQL	Recomendado por el docente; integra Git y GitHub
Git 2.53.0	Control de versiones, ramas y etiquetas	Estándar; ramas por issue
GitHub	Repositorio, issues y pull requests	Sugerido por el docente; cierra issues con commits
PHP 8.2.12	Controladores, lógica, validaciones, datos	Visto en el curso; sesiones y excepciones
XAMPP 8.2.12	Apache, PHP y MariaDB locales	Gratuito y sin configuración manual
Apache 2.4.58	Servidor web de la carpeta public	Incluido en XAMPP
MariaDB 10.4.32	Motor de base de datos	Gratuito, compatible con MySQL
PDO	Conexión y consultas	Consultas preparadas contra inyección SQL

Herramienta	Uso	Motivo
Composer 2.10.2	Dependencias PHP	Instalar librerías
vlucas/phpdotenv 5.6.4	Cargar credenciales desde .env	No subir contraseñas al repositorio
JavaScript, HTML5, CSS3	Páginas, estilos y llamadas fetch	Vistos en el curso; código simple
Figma	Prototipos de pantallas	Simple y ya conocido

Librerías con ejemplo:

- **phpdotenv:** config/config.php carga el .env; en el repositorio solo está .env.example con BD_HOST, BD_PORT, BD_USER, BD_PASS y BD_NAME.
- **PDO:** en DAOEquipo::listar el filtro usa parámetros (e.estado = :estado) con prepare y execute; el dato no se concatena al SQL.
- **fetch:** api.js centraliza las llamadas con llamarApi y agrega el token CSRF en los métodos que modifican datos.

5.2 ESTÁNDARES DE CODIFICACIÓN

Basados en *Código limpio* de Robert C. Martin:

- **Nombres:** clases sustantivo, métodos verbo (DAOEquipo: listar, obtener, crear). Clases como "Controlador + recurso". Archivos PHP en PascalCase; variables y métodos en camelCase y en español; tablas en mayúsculas y columnas en camelCase; prefijos solo pk_ y fk_; JS y CSS en camelCase.
- **Comentarios:** todo método público con PHPDoc (@param, @return, @throws); comentarios en línea explican el porqué; prohibido dejar código comentado.
- **Errores:** los controladores lanzan excepciones con código HTTP; index.php las captura y las pasa a ManejadorErrores; las operaciones de varios pasos usan rollBack.

5.3 REPOSITORIO, RAMAS Y VERSIONADO

GitHub se eligió por repositorios privados, issues, pull requests, cierre por palabras clave, historial visible e integración con VS Code. Desventajas: depende de un servicio externo y de una cuenta personal, no de una organización.

Ramas: variante simplificada de GitHub Flow. main es la rama estable y solo recibe pull requests (salvo documentación). Cada issue tiene su rama, que parte de main; los cambios chicos usan ramas temporales.

Commits: prefijos feat (funcionalidad), fix (corrección) y docs (documentación, únicos admitidos directo en main). Se usan Closes, Fixes o Resolves para cerrar issues al mergear. Mensaje en imperativo; un commit agrupa cambios de una sola cosa y debe entenderse sin abrir el código.

Versionado semántico Mayor.Menor.Parche: Mayor al estar lista la primera entrega, Menor al agregar funcionalidad compatible, Parche ante correcciones menores.

Estructura de carpetas: baseDeDatos, docs y PROYECTO; dentro de PROYECTO: app (controlador, i18n, modelo, núcleo, rutas, vista), config, public (assets con css, img y js; paginas), vendor y archivos .env, .env.example, .gitignore y README.

5.4 PERSISTENCIA Y NORMALIZACIÓN

La base es relacional (MySQL/MariaDB), controlada desde PHP. Las 16 tablas quedaron en tercera forma normal:

Tabla	Clave primaria	Claves foráneas	Observación de 3FN
EQUIPO	idEquipo	idLaboratorio	El estado no determina la disponibilidad (un préstamo la cambia)
PRESTAMO	idPrestamo	idEquipo, cedulaSolicitante	Estado derivado de la fecha real de devolución: redundancia controlada, se actualizan juntos

Tabla	Clave primaria	Claves foráneas	Observación de 3FN
USUARIO	cedula	—	Nombre y apellido independientes
ADMINISTRADOR, TECNICO, DOCENTE	cedula	cedula → USUARIO	Un solo atributo por tabla
LABORATORIO	idLaboratorio	—	El estado cambia en el tiempo, no depende del número
SOLICITUD_LABORATORIO	idSolicitud	idLaboratorio, cedulaSolicitante → DOCENTE	Fecha estimada elegida por el docente
TICKET	idTicket	laboratorio, equipo	Guarda el laboratorio aunque el equipo se mueva después
DIAGNOSTICO	idDiagnostico	idTicket, cedulaTecnico	Varios diagnósticos por ticket
SOLUCION	idSolucion	idDiagnostico, cedulaTecnico	El técnico puede ser distinto del diagnóstico
REPARACION	idReparacion	idDiagnostico, cedulaTecnico	Corregida: se quitaron idTicket e idEquipo
HISTORIAL_ESTADO_EQUIPO	idHistorialEstado	idEquipo, cedulaUsuario	Cada fila es un cambio único
MOVIMIENTO_EQUIPO	idMovimiento	idEquipo, laboratorio anterior y nuevo, cedulaUsuario	Cada movimiento con su origen

Tabla	Clave primaria	Claves foráneas	Observación de 3FN
INTERVENCION	idIntervencion	idEquipo, cedulaTecnico	Descripción libre por intervención
REEMPLAZO	idReemplazo	idEquipo, cedulaTecnico	Componentes distintos en el tiempo

El caso destacado es **REPARACION**: el modelo inicial tenía idDiagnostico, idTicket e idEquipo. Como diagnóstico determina ticket y ticket determina equipo, había una dependencia transitiva que podía generar contradicciones. Se eliminaron idTicket e idEquipo y se obtienen uniendo REPARACION, DIAGNOSTICO y TICKET.

5.5 PATRONES DE DISEÑO

Patrón	Qué resuelve	Ubicación
DAO	Separa el SQL; los controladores solo piden listar, obtener o crear	app/modelo/DAO*.php
Front Controller	Un único punto de entrada para todas las solicitudes, sesión y errores	public/index.php y app/rutas/api.php
MVC	Separa datos, lógica y presentación	app/modelo, app/controlador, app/vista/RespuestaJson.php y public

Ejemplos del original: el DAO recibe la conexión PDO en el constructor; DAOEquipo::obtener usa un JOIN con LABORATORIO y el parámetro :idEquipo; api.php mapea rutas como "equipos" o "tickets" a su controlador y método; el controlador crea su DAO y responde con RespuestaJson::exito.

5.6 API

La API es la única vía entre cliente y servidor. Recibe todo por public/index.php?ruta=nombre, valida sesión, rol y token CSRF, valida los datos, usa los DAO y responde siempre en JSON.

Ruta	Métodos	Acceso
login	POST	Público
logout, sesion	POST, GET	Sesión iniciada
idioma, traducciones	POST, GET	Sin restricción de rol
usuarios	GET, POST, PUT, DELETE	Coordinador
laboratorios, equipos	GET, POST, PUT, DELETE	Lectura: todos los roles; resto: coordinador y técnico
prestamos	GET, POST, PUT, DELETE	Coordinador y técnico
tickets, solicitudes	GET, POST, PUT, DELETE	Según el método
diagnosticos, reparaciones, soluciones	GET, POST, PUT, DELETE	Alta: técnico; resto: coordinador y técnico
intervenciones, reemplazos	POST	Técnico
historialEquipo, dashboard	GET	Coordinador y técnico

Formato de respuesta: status (success o error), message y data (null en los errores).
Los mensajes vienen en el idioma de la sesión. Las solicitudes con cuerpo usan application/json y, si modifican datos, el encabezado X-CSRF-Token.

Código	Cuándo
200	Consulta o modificación correcta
201	Recurso creado
400	Datos inválidos o referencia inexistente
401	Sin sesión o credenciales incorrectas
403	Rol incorrecto o token CSRF inválido
404	Recurso o ruta no encontrados
405	Método no permitido

Código	Cuándo
409	Duplicado o no se puede eliminar por datos relacionados
500	Error interno; se registra y se muestra un mensaje genérico

Ejemplos: GET equipos&id=PC-01 devuelve 200 con los datos del equipo (Dell, Lab 1, Funcionando, Disponible) o 404 si no existe. POST equipos con el JSON del equipo devuelve 201, o 409 si el ID ya existe.

Cliente: la función llamarApi arma la URL, agrega el token CSRF desde sessionStorage solo si no es GET, envía JSON y convierte los errores HTTP en excepciones; ante un 401 vuelve al login.

Criterios REST: cumple recursos con sustantivos, verbos HTTP correctos, códigos adecuados, JSON uniforme y GET sin efectos secundarios. No cumple rutas jerárquicas ni ausencia de estado (usa sesión).

5.7 REPOSITORIO Y ANEXOS DE CÓDIGO

El repositorio es privado y requiere acceso. Issues destacados:

Issue	Commit	Mensaje
#3 Registrar equipos	ad2a4d0	feat(Equipos): registrar equipos en una tabla
#38 Registrar diagnósticos	a1d8e9a	Registrar Diagnóstico
#21 Crear tickets	b939338	Crear Ticket RF19
#48 Iniciar sesión	b81435d	fix(login): el error de contraseña empujaba otras etiquetas
#154 Alta de usuarios y roles	36c0536	Implementación de usuarios y roles
#164 Equipos desde la base	ce15e12	Base de datos en equipos
#209 Interfaz bilingüe	4dc145f	Interfaz Bilingüe
#210 Préstamos	3210ab6	Módulo de préstamos y devoluciones
#213 Parámetros en crearUsuario	4bcf211	Corregir alta de usuarios

El listado completo va del #3 al #221: los primeros siguen los requerimientos funcionales; del #140 al #177 la integración con la base; del #200 al #221 la API REST, fetch, consultas preparadas, bilingüe, préstamos, trazabilidad y las correcciones pedidas por el docente (excepciones, validación uniforme, claves foráneas, normalización, separación de responsabilidades y patrones).

6. CIBERSEGURIDAD

Se identificaron cinco ataques posibles según la estructura de la aplicación, con un método de protección y otro de detección para cada uno.

Ataque	Protección	Detección
Inyección SQL	Consultas preparadas, sin concatenar variables	Revisar logs de PHP y Apache; probar "OR 1=1" en login y buscadores; muchos errores seguidos indican un ataque
Fuerza bruta al login	Contraseñas cifradas con hash y límite de intentos por sesión	Contar intentos fallidos por usuario o IP con fecha y hora
CSRF	Token aleatorio por sesión validado en cada formulario que modifica datos	Registrar tokens ausentes o que no coinciden; muchos indican solicitudes externas
XSS	Función de PHP que convierte caracteres especiales (<, >, ", ') en texto inofensivo	Probar etiquetas script en campos de texto y revisar la base en busca de ellas
Exposición de datos sensibles	Credenciales en .env con phpdotenv; solo se expone la carpeta pública	Intentar abrir .env desde el navegador; verificar .gitignore y que los errores no muestren rutas ni datos

Se elaboró además un mapa conceptual de amenazas y medidas.

6.1 POLÍTICAS DE SEGURIDAD DE DEKLAN ENTERPRISE

1. **Autenticación y contraseñas:** hash, nunca texto plano; límite de intentos.
2. **Tratamiento de datos:** sentencias preparadas y sanitización antes de mostrar.

3. **Validación de formularios:** token CSRF validado por el servidor.
4. **Gestión de credenciales:** variables de entorno en .env excluido del repositorio; solo la carpeta pública es accesible.
5. **Registro y monitoreo:** errores e intentos fallidos con fecha y hora, revisados periódicamente.

6.2 MARCO NORMATIVO URUGUAYO

- **Ley 18.331 de Protección de Datos Personales:** su artículo 10 obliga a garantizar la seguridad y confidencialidad de los datos, evitando adulteración, pérdida o acceso no autorizado.
- **Ley 19.670:** refuerza la obligación de aplicar medidas de seguridad y responder por su cumplimiento.
- **URCDP:** organismo que supervisa la ley y puede sancionar su incumplimiento.

7. CONCLUSIONES GENERALES

El SGRSI resolvió el problema de origen: cada incidencia tiene número, estado y responsable, y cada equipo su historial. El docente conoce el estado de su pedido, el técnico organiza su trabajo por prioridad y la coordinación cuenta con datos para decidir reparaciones, reemplazos y compras.

Se cumplieron los objetivos: todos los módulos de la letra, acceso por roles, dos idiomas, diseño adaptable, 52 casos de prueba aprobados, validación con usuarios y una inversión recuperable dentro del período evaluado.

Pendientes a futuro:

- Avisos cuando un ticket supera su fecha límite.
- Textos de ayuda en formularios y un flujo del técnico más claro.
- Pruebas con docentes y técnicos reales en el uso diario.
- Pruebas con muchos usuarios simultáneos.

7.1 REFLEXIÓN FINAL DEL EQUIPO

- **Aprendizaje:** un sistema no se construye solo programando; primero hay que escuchar a los usuarios, convertir lo relevado en requerimientos y validar. Las devoluciones docentes sirven para mejorar, no solo como nota.
- **Dificultades:** mala organización al inicio, superposición con otras materias, primera entrega incompleta en UTULAB y temas nuevos como la API.
- **Soluciones:** revisar letra y rúbrica, Kanban en Trello, referentes por materia, fechas internas, WhatsApp, reuniones virtuales con actas y GitHub para seguir el código.
- **Decisiones que cambiaron:** cascada a incremental, prototipo rehecho, roles renombrados, menú lateral a barra superior y préstamos postergados.
- **Si empezaran de nuevo:** organizar los tiempos desde la primera semana, empezar antes el código y mantener el prototipo actualizado junto con el producto.

Reflexiones individuales finales:

- **Julián González (Coordinador):** su rol fue corregir y supervisar; lo disfrutó y lo repetiría, quizás con otros perfiles de compañeros.
- **Santiago Cuello (Subcoordinador):** aprendió a adaptarse a cambios, a manejar los tiempos y a convivir con el equipo.
- **Ezequiel Forcelledo (Integrante 1):** proyecto algo estresante pero en un entorno amigable; a mejorar los tiempos y el cumplimiento del reglamento; considera que hizo bien la documentación de UTULAB.
- **Federico Venis (Integrante 2):** le costó llegar a tiempo, pero le dejó aprendizajes sobre trabajo en equipo y organización.

8. GLOSARIO

Término	Definición
API	Rutas que comunican las pantallas con el servidor

Término	Definición
Backend / Frontend	Parte en el servidor / parte que ve el usuario en el navegador
Base de datos relacional	Información en tablas relacionadas
Consulta preparada	Separa los datos del usuario del código SQL
Contenedor	Entorno aislado con Docker con todo lo necesario para ejecutar una aplicación
CRUD	Crear, leer, actualizar y eliminar
CSRF	Ataque que usa una sesión iniciada para enviar solicitudes falsas
DAO	Clase que accede a la base para una parte del sistema
Dashboard	Panel de métricas resumidas
DDL / DML	SQL de estructura / SQL de datos
Endpoint	Dirección de la API que recibe una petición
Fetch	Función JS para enviar y recibir datos sin recargar
Intranet	Red interna de una institución
Inyección SQL	Insertar SQL por formularios para leer o alterar datos
JSON	Formato de texto para intercambiar datos
Kanban	Tareas en columnas según su estado
Mesa de ayuda	Servicio que recibe y sigue pedidos de soporte
MVC	Modelo, vista y controlador
Prototipo	Versión visual para mostrar y probar el sistema
Puntos de función	Métrica de tamaño según funciones para el usuario
Repositorio	Lugar del código con su historial

Término	Definición
Responsivo	Se adapta al tamaño de pantalla
Rol	Perfil que define qué puede hacer un usuario
Ticket	Incidencia con número, estado y prioridad
Trazabilidad	Seguir la historia de un equipo o pedido
XSS	Insertar código que se ejecuta en el navegador de otros

9. BIBLIOGRAFÍA Y ANEXOS

Bibliografía citada en el original: Trello (Atlassian), documentación de Ubuntu Server (Canonical), Composer, el repositorio ProyectoFinal en GitHub, el prototipo en Figma, Docker Compose, Google Forms, la letra del Proyecto 2026 del ITI, draw.io, phpdotenv, los PDF sobre API del docente Leandro Lopez, el manual de MySQL 8.0, el manual de PHP y herramientas de IA (Claude, ChatGPT, Gemini) para comprensión.

Anexos:

- **A – Actas de reuniones 1 a 10:** decisiones y tareas asignadas; respaldan la organización del equipo.
- **B – Devolución de la primera entrega:** devolución de UTULAB y planilla de correcciones; explican los cambios de la solución.
- **C – Recursos externos:** enlaces al repositorio de GitHub y al prototipo en Figma con sus 26 pantallas (con una imagen alternativa en el repositorio si Figma no abre).